Response to P4043R0

Document Number:	D4045R0
Date:	2026-03-08
Audience:	EWG

Reply-to:	Vinnie Falco vinnie.falco@gmail.com

Table of Contents

Abstract

Revision History

R0: March 2026 (Croydon)

1. Disclosure

2. The Question

3. The Evidence

4. The Record

5. Framing

6. The Paradox

7. Procedure

References

Abstract

P4043R0[1] ("Are C++ Contracts Ready to Ship in C++26?") proposes that EWG reconsider the shipping vehicle for C++ Contracts. Every paper it cites, every concern it raises, and every alternative it suggests was available to the committee before the most recent decisions on the topic.

Revision History

R0: March 2026 (Croydon)

- Initial version.
- Corrected imprecise ballot characterization in Section 4 ("none has achieved consensus" replaced with "each was rejected") and added Section 5 on the distinction between consensus against and no consensus.

---

1. Disclosure

The author has no involvement in the C++ Contracts facility and holds no position on its design. The question addressed here is narrow: does P4043R0[1] present information that the committee has not already considered?

---

2. The Question

P4043R0[1] asks whether EWG should reconsider the shipping vehicle for C++ Contracts. The paper asks this question seven times:

Section	Representative statement
1	"This paper therefore proposes a simple question for EWG: whether the C++ Contracts facility, in its current form, is ready to ship in C++26."
2	"The presence of continuing NB discussion suggests that further design exploration may still be beneficial."
3	"Taken together, this body of work indicates that the design space for Contracts continues to be actively discussed."
4	"If substantial questions remain open, deferring the feature could allow the committee to incorporate additional design insights."
5	"If the committee concludes that the C++ Contracts facility would benefit from additional design time, several alternative paths could be considered."
6	"Do we believe that the C++ Contracts facility is ready to ship in C++26?"
7	"Deferring the feature could allow WG21 additional time to refine the design."

3. The Evidence

P4043R0[1] cites 15 papers. The following table lists each one with its publication date, drawn from the paper's own reference section:

Paper	Published
P2900R14	2025-02-13
P3573R0	2025-01-12
P3829R0	2025-07-15
P3835R0	2025-09-03
P3851R0	2025-09-29
P3853R0	2025-10-05
P3909R0	2025-11-02
P3911R2	2026-01-14
P3912R0	2025-12-15
P3919R0	2025-11-05
P3946R0	2025-12-14
P4005R0	2026-02-02
P4009R0	2026-02-09
P4015R0	2026-02-10
P4020R0	2026-02-23

Every paper in this list was published before P4043R0[1] and was available at prior committee meetings. The paper does not cite any new implementation result, deployment observation, or design analysis.

4. The Record

On 2025-02-11, EWG polled on whether to remove P2900[2] (“Contracts for C++”) from consideration for C++26 and find a different shipping vehicle. The result was consensus against (P2899R1[3]).

On 2025-02-13, LWG forwarded P2900R14[2] to plenary with consensus (P2899R1[3]).

On 2025-02-16, plenary adopted P2900R14[2] into the C++26 Working Paper with strong consensus (P2899R1[3]).

P2900R14[2] (published 2025-02-13) remains the latest revision. The changes applied since adoption - the removal of `evaluation_exception()` via P3819R0[9] and the addition of a feature test macro via P3886R0[10] - are NB comment resolutions that narrow or clarify the facility. The core design is unchanged. Multiple proposals to modify the design have been presented since adoption; each was rejected (P3846R0[4]).

`constexpr` shipped as a minimal language feature in C++11 (N2235[5]). C++14 relaxed its constraints dramatically (N3652[6]). The MVP model - ship a minimal language feature in one standard, expand it in the next - is established WG21 practice.

---

5. Framing

A committee poll produces one of three outcomes: consensus for, consensus against, or no consensus. These are not interchangeable. Consensus against a proposal is an affirmative decision - the committee considered the question and answered it. No consensus means the committee did not reach a decision in either direction.

P4043R0[1] describes prior ballot outcomes without distinguishing between these cases. The EWG poll on removing Contracts from C++26 was not “no consensus.” It was consensus against removal. The proposals to modify the adopted design were not merely unsuccessful - they were rejected. Characterizing consensus against as the absence of consensus erases the committee’s decision.

---

6. The Paradox

P4043R0[1] recommends deferral to allow “implementation and deployment experience.” Language features do not receive widespread deployment outside standards. The Concepts TS (ISO/IEC TS 19217:2015[7]) demonstrated this: the TS was withdrawn, and Concepts were redesigned and shipped directly in C++20. P4043R0[1] prescribes the condition it claims to remedy.

7. Procedure

P4043R0[1] (dated 2026-03-07) does not appear in any official WG21 mailing. The 2026-02 pre-Croydon mailing (released 2026-02-23) contains papers through P4032R0[8]. The paper presents no proposed solution to the one outstanding NB comment (RO 2-056).

References

1. P4043R0 - "Are C++ Contracts Ready to Ship in C++26?" (Darius Neaţu, 2026). https://wg21.link/p4043r0
2. P2900R14 - "Contracts for C++" (Joshua Berne, Timur Doumler, Andrzej Krzemieński, et al., 2025). https://wg21.link/p2900r14
3. P2899R1 - "Contracts for C++ - Rationale" (Timur Doumler, Joshua Berne, Andrzej Krzemieński, Rostislav Khlebnikov, 2025). https://wg21.link/p2899r1
4. P3846R0 - "C++26 Contracts, reasserted" (Timur Doumler, Joshua Berne, 2025). https://wg21.link/p3846r0
5. N2235 - "Generalized Constant Expressions" (Gabriel Dos Reis, Bjarne Stroustrup, Jens Maurer, 2007). https://open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2235.pdf
6. N3652 - "Relaxing constraints on constexpr functions" (Richard Smith, 2013). https://open-std.org/jtc1/sc22/wg21/docs/papers/2013/n3652.html
7. ISO/IEC TS 19217:2015 - "C++ Extensions for Concepts" (ISO, 2015). Withdrawn.
8. P4032R0 - last paper in the 2026-02 pre-Croydon mailing.
9. P3819R0 - "Remove evaluation_exception from Contracts" (Timur Doumler, Joshua Berne, 2025). https://wg21.link/p3819r0
10. P3886R0 - "Contract violation handler replaceability" (Timur Doumler, Joshua Berne, 2025). https://wg21.link/p3886r0
11. P3573R0 - "Contract concerns" (Michael Hava, J. Daniel Garcia Sanchez, et al., 2025). https://wg21.link/p3573r0
12. P3829R0 - "Contracts do not belong in the language" (David Chisnall, John Spicer, et al., 2025). https://wg21.link/p3829r0
13. P3835R0 - "Contracts make C++ less safe - full stop!" (John Spicer, Ville Voutilainen, Jose Daniel Garcia Sanchez, 2025). https://wg21.link/p3835r0
14. P3851R0 - "Position on contract assertions for C++26" (J. Daniel Garcia, et al., 2025). https://wg21.link/p3851r0
15. P3853R0 - "A thesis+antithesis=synthesis rumination on Contracts" (Ville Voutilainen, 2025). https://wg21.link/p3853r0

16. P3909R0 - ‘‘Contracts should go into a White Paper’’ (Ville Voutilainen, 2025). https://wg21.link/p3909r0
17. P3911R2 - ‘‘Make Contracts Reliably Non-Ignorable’’ (Darius Neaţu, Andrei Alexandrescu, et al., 2026). https://wg21.link/p3911r2
18. P3912R0 - ‘‘Design considerations for always-enforced contract assertions’’ (Timur Doumler, et al., 2025). https://wg21.link/p3912r0
19. P3919R0 - ‘‘Guaranteed-(quick-)enforced contracts’’ (Ville Voutilainen, 2025). https://wg21.link/p3919r0
20. P3946R0 - ‘‘Designing enforced assertions’’ (Andrzej Krzemieński, 2025). https://wg21.link/p3946r0
21. P4005R0 - ‘‘A proposal for guaranteed-(quick-)enforced contracts’’ (Ville Voutilainen, 2026). https://wg21.link/p4005r0
22. P4009R0 - ‘‘A proposal for solving all of the contracts concerns’’ (Ville Voutilainen, 2026). https://wg21.link/p4009r0
23. P4015R0 - ‘‘Enforcing Contract Conditions with Statements’’ (Lisa Lippincott, 2026). https://wg21.link/p4015r0
24. P4020R0 - ‘‘Concerns about contract assertions’’ (Andrzej Krzemieński, 2026). https://wg21.link/p4020r0